SWARM INTELLIGENCE SIMULATION FOR AUTONOMOUS DRONES

Development of Hardware-Independent Swarm Intelligence and Obstacle Avoidance Simulations

Göktuğ Arda Gök

30975

Internship Dates:

16.06.2025 - 11.08.2025

Internship Company:

altınay Savunma

Supervised by:

Adnan Kefal

Sabancı Üniversitesi

Faculty of Engineering and Natural Sciences

Mechatronics Engineering

II. ABSTRACT

Altınay Savunma is a technology company in Türkiye that mainly works in defense and aerospace projects. The company develops different critical systems such as robotics, automation, and unmanned vehicles.

In this internship I focused on swarm simulations for autonomous aerial systems. First, I created a user interface in MATLAB App Designer to control the swarm. With this simulation I tested different formations, obstacle scenarios, and attack commands. The movement of the drones was based on the Artificial Potential Field (APF) method, and I also added a “Danger Zone” escape logic to handle situations when drones get too close to obstacles. A simple state machine controlled the general behavior of the swarm with states like Idle, Flying, Moving, and Attacking.

After completing the MATLAB part, I moved the project into the ROS 2 environment and used RViz for visualization. In this step, the swarm and obstacles were shown with markers, and the same logic was tested in a robotics framework. The results showed that the swarm could keep its formation, avoid obstacles, and complete coordinated maneuvers in different cases.

This experience helped me see how swarm algorithms can first be tested in MATLAB and then adapted into a robotics system like ROS 2. As a next step, the system can be improved by testing with real hardware and making the obstacle avoidance more efficient.

Table of Contents

1. Introduction

This internship report summarizes the work I have done at Altınay Savunma during the summer period. The main focus of the internship was on swarm technologies and how to simulate autonomous drone coordination in different environments. Simulation is an important step before moving to hardware, because it allows testing many scenarios in a safe and low-cost way.

The project aimed to design and implement swarm algorithms that can control a group of drones, manage their formations, and make them avoid obstacles while completing tasks. For this purpose, I first worked in MATLAB App Designer to build an interface where swarm behaviors could be tested. Later, the same ideas were moved into ROS 2 and visualized in RViz, which is a more realistic robotics framework.

The report is organized as follows:

- Section 2 introduces the company, Altınay Savunma, and gives information about its background and activities.

- Section 3 explains the background of the project, the department I worked in, the motivation, and related literature.

- Section 4 describes the internship project in detail, including objectives, my responsibilities, the methods used, and the results.

- Section 5 discusses my internship experience, what I learned, and the difficulties I faced.

- Section 6 concludes the report, and Section 7 gives recommendations for future students.

2. Company Information

Altınay Defense Technologies Inc. (Altınay Savunma Teknolojileri A.Ş.) is located in Teknopark Istanbul, Pendik, Istanbul. The company's official address is Teknopark Bulvarı No:1/4A, İç Kapı No:201–202, Sanayi Mahallesi, Pendik, Istanbul. The main contact phone number is +90 (216) 504 00 50 and the company website is www.altinaysavunma.com.

Altınay Defense Technologies was founded in 2014 as part of the Altınay Technology Group, which itself has a long history in robotics and automation since the early 1990s. From its establishment, the defense company has been working on critical systems for motion control, unmanned vehicles, weapon systems, and production technologies that meet national needs in the defense and aerospace sector.

The company's main facilities are located in Teknopark Istanbul, where it carries out its research and development, production, and integration activities. The total area of facilities, including production halls, offices, and laboratories, is reported to be more than 40,000 square meters. In these facilities, the company develops motion control systems such as actuators and gimbals, weapon pedestals, as well as integrated defense subsystems and production technologies.

Altınay Defense is a subsidiary of Altınay Technology Group, which also includes other companies such as Altınay Robotics, Altınay Electromobility, Altınay ModCenter, TAAC

Aviation Technologies, and DASAL Aerospace Technologies. Through these partners, Altınay has access to broader expertise in aviation, robotics, and system integration.

Within the defense industry, Altınay competes with other major Turkish companies such as Aselsan, Roketsan, Havelsan, and STM, which also develop critical defense technologies. However, Altınay differentiates itself through its strong background in motion control, automation, and specialized mechanical systems. Its main customers are Turkish defense institutions, government agencies, and private sector partners. The company also works with various suppliers for electronic, mechanical, and material components required in system integration.

In terms of scale, Altınay Defense employs more than 200 people, the majority of whom are engineers. Some public sources report this number as around 300 in recent years, which shows the steady growth of the company. The organizational structure typically includes a general management office, research and development, production and assembly, systems integration and software, quality and certification, and project management departments. Each department contributes to the life cycle of defense projects, from design to testing and final delivery.

Overall, Altınay Defense Technologies has become one of the important companies in Türkiye's defense and aerospace sector. Its focus on critical national technologies, strong engineering capacity, and wide range of projects make it a significant environment for research and development activities, including the internship project described in this report.

Figure 1. Engineering Department of Altınay Defense at Teknopark

3. Project Background

3.1 Department Information

During my internship I was officially placed in the Mechanical Department of Altınay Defense. This department mainly focuses on the design, development, and testing of mechanical subsystems such as actuators, pedestals, and structural parts of unmanned platforms. My desk and daily guidance were here, and I had access to their design and analysis tools.

However, because the project was related to swarm simulations, I also had strong interactions with other departments. The Control Department is responsible for developing flight control strategies, stability systems, and swarm coordination algorithms. The Artificial Intelligence Department works on decision-making, autonomy, and behavior logic, applying methods such as state machines and navigation algorithms. I also interacted with the Software

Department, which is in charge of implementation in programming languages, middleware integration, and ROS based testing.

My work was mainly a bridge between these groups. The mechanical background gave me an understanding of the physical constraints, while the control and AI teams guided me in algorithm development. The software team supported the implementation and testing in ROS 2 and RViz. This cross-departmental collaboration gave me the chance to see how different engineering areas combine to develop advanced unmanned systems.

I have worked with:

Davut Bakan - Head of Mechanical Department - davut.bakan@altinay.com

Gürcan Özdemir - Mechanical Engineer - gurcan.ozdemir@altinay.com

3.2 Status Before Project

Before I started my internship, the company already had a swarm drone system that was tested in real flight conditions. These tests gave valuable results, but my role was not to join the field activities. Instead, the expectation was to design a virtual environment that could represent the swarm in a flexible way.

At that time, there was no digital platform inside the company where both the swarm and the environment could be controlled together. Engineers did not have a tool where they could quickly set the number of drones, change their formation, add obstacles, or introduce enemy targets in a simple interface. Because of this, most tests had to be repeated in real life even for small changes, which is not always practical.

The main idea of my project was therefore to provide such a simulation environment. It would allow engineers to create different test cases on the computer, see the swarm response, and visualize the outcomes before going to the field. This would not replace real tests, but it would make them easier to prepare and analyze. In addition, the virtual environment could also be used for demonstration purposes, for example when presenting swarm capabilities to colleagues or supervisors.

3.3 Motivation / Problem Definition

The main motivation of this project was the company's request to build a virtual environment for swarm drones. They already had working drones and had done tests, but there was no software where the swarm and the environment could be controlled together on a computer. The company wanted an additional digital tool to support their existing work by allowing test cases to be created, obstacles or enemy targets to be placed, and swarm behavior to be visualized directly.

The problem can be described simply: there was no dedicated simulation platform inside the company for this purpose. Engineers could not easily adjust the number of drones, change formations, or introduce new conditions virtually.

The project was therefore defined as the development of a software tool where swarm drones can be represented in a virtual environment. The tool was expected to include functions for swarm control, environment setup, and visualization. It would support the company's existing efforts by making test cases easier to design, analyze, and present.

3.4 Related Literature

Swarm drones and swarm robotics are topics that many researchers and companies are working on today. The main idea is to use many small drones together instead of one big system. This can be more flexible, cheaper, and sometimes even more reliable. To control swarms, different mathematical and algorithmic methods are used. In my project I focused on simulation, so I needed to look at methods that are practical and can run in real time.

One of the main methods that I used was the Artificial Potential Field (APF) approach. In APF, the drone feels an attraction force toward its goal and a repulsion force from obstacles. The calculation is simple: it is based on the distance between the drone, the goal, and the obstacles. Because it is simple, it is a popular method in swarm robotics. But it also has some problems, such as drones getting stuck in local minima. Some studies propose improvements, like adding extra forces when the drone is too close to an obstacle. In my project I also added a “danger zone” idea, where the repulsive force becomes stronger to help the drone escape.

In the paper by Tang et al. (2025), the authors presented a swarm algorithm for patrolling durian orchards. Their focus was on making sure drones can cover a wide area without overlapping paths. I found this useful because it showed how coordination can be used in a real application, in this case agriculture. It made me realize that swarm methods are not only for defense, but also for many civil areas.

Marek et al. (2025) worked on collision avoidance for drone swarms. They discussed the importance of drones avoiding each other as well as obstacles. This was important for me, because in my simulation I also had to make sure drones did not crash when flying close together. Their paper gave me an idea of how researchers design and test avoidance methods in simulation before trying them in reality.

From the industry side, I read the STM ThinkTech report (2020) on swarm UAVs. This report talks about how swarms can change modern defense by being more autonomous and harder to counter. For me, the most interesting part was that they emphasized the importance of testing and simulation before using such systems in the field. This is directly related to what I tried to do during my internship: building a tool that lets engineers try swarm scenarios virtually.

Besides these main references, I also used my general knowledge from robotics and control courses. For example, I used a state machine to control the swarm behavior with states like Idle, Flying, Moving, and Attacking. This is a common method in robotics textbooks, and I saw it was also helpful in my project.

To sum up, the literature shows that APF is a common method for navigation, that coordination and collision avoidance are important topics, and that industry also sees swarms as a strategic technology. The academic papers by Tang et al. (2025) and Marek et al. (2025) gave me ideas for my implementation, while the STM (2020) report gave me context for why companies care about this topic.

4. Internship Project

4.1 Project Objective

The main objective of this project was to design and implement a virtual environment for swarm drones. The company already had working swarm systems, but there was no software tool where both the swarm and the environment could be controlled together. The project aimed to provide such a platform, mainly for testing and visualization.

The boundaries of the project were clearly defined. The scope only covered the simulation side of swarm behavior. My task was not related to hardware design, manufacturing, or joining field flight tests. Instead, I focused on creating a digital tool where the number of drones, their formations, enemy targets, and obstacles could be changed easily, and the swarm's reaction could be observed.

The project addresses the problem by offering a software-based solution. With this tool, engineers can test scenarios virtually, prepare cases before going to the field, and also use the environment for presentation purposes. It does not replace real flights, but it supports them by making it easier to design and analyze test cases in advance.

4.2 My Responsibilities

I worked alone on this project, so all the responsibilities and tasks were my own. I was in charge of designing the simulation logic, building the interface, and connecting the system to visualization platforms.

The main activities I carried out were:

Developing a user interface in MATLAB App Designer where swarm drones could be controlled.

Implementing Artificial Potential Field (APF) navigation and a “danger zone” escape logic for obstacle avoidance.

Designing a state machine to manage swarm behavior such as Idle, Flying, Moving, and Attacking.

Running simulations in MATLAB to test formations, obstacle scenarios, and attack maneuvers.

Extending the project to ROS 2 and RViz, so that swarm behavior could be visualized with markers in a robotics framework.

Writing and documenting the codes so they could be reused and improved later.

Since I worked alone, there were no direct team members in my project. However, I received guidance from engineers in the Mechanical, Control, AI, and Software departments whenever I needed feedback or information.

4.3 Methodology / Tools

For this project, I mainly worked with MATLAB App Designer and ROS 2 with RViz.

MATLAB App Designer was used for building the first simulation interface, because it allowed me to create buttons, sliders, and visual plots quickly. With this interface, I could set the number of drones, choose formations, add obstacles, and run test cases in a controlled way.

The core algorithm for navigation was based on the APF method. In this approach, drones are pulled toward their goals by attractive forces, and pushed away from obstacles by repulsive forces. I added a “danger zone” rule to make drones react more strongly when they are too close to an obstacle. I also included a state machine to manage swarm behavior, with states like Idle, Flying, Moving, and Attacking. This made the simulation more realistic and organized.

After completing the MATLAB stage, I extended the work into ROS 2, which is a robotics middleware. Here I used RViz for visualization. Drones, obstacles, and the environment were represented by markers, and their movements were updated in real time during the simulation. This step was important because ROS 2 is commonly used in robotics projects, and it made the tool more relevant for future development.

4.4 Expected Outcomes & Deliverables

The expected outcome of this project was a working simulation environment for swarm drones. The company wanted a tool where both the swarm and the environment could be controlled virtually, so that different cases could be tested and visualized. The main objective was to make it easier to design and analyze swarm behavior before going to the field.

To achieve this objective, I created two stages of deliverables. The first was the MATLAB App Designer interface, which allowed quick control of drones, formations, and obstacles. The second was the ROS 2 and RViz visualization setup, which showed the swarm and the environment inside a robotics framework.

In addition to these software deliverables, the project also produced a technical report explaining the algorithms used, and documented codes that can be improved later by other engineers.

4.5 Details

4.5.1 MATLAB Simulation Environment

I started my project by developing a simulation in MATLAB App Designer. The reason for using App Designer was that it allowed me to build an interactive user interface where I could control the swarm with buttons, sliders, and input fields. In the app I initialized a grid of 100 drones, and then I could take off, land, apply different formations, move the swarm center, or send some of the drones to attack an enemy.

The app had a number of components. There were buttons for Takeoff, Land, Home, Apply Formation, Move to Coordinates, Set Enemy, and Attack. There were dropdown menus to choose between formations (circle, square, semi-circle, and v-shape) and to select obstacle scenarios (none, choke point, scattered field, and wall). Numeric input fields allowed me to set target coordinates and enemy positions. A slider was used to select the percentage of drones for the attack. In addition, I added indicators: a lamp that showed the current state of the swarm with different colors (green for idle, blue for flying, yellow for moving, red for attacking, orange for resetting) and a label that showed the number of drones still active after an attack. The app also had two plots: one main map where drones, obstacles, and enemies were shown, and a second telemetry plot where I tracked the formation error over time.

The drones moved in a loop that updated every 50 milliseconds. For each drone, the movement was calculated using a combination of forces. The drone was always pulled toward its assigned target, but when it was close to an obstacle it was pushed away by a repulsive force and also received a sideways push (perpendicular force). If the drone was inside a danger zone, these forces were increased so that the drone could escape faster. This made the drones slide around obstacles instead of getting stuck.

Formations were recalculated around the swarm center whenever the user pressed “Apply Formation.” The options were circle, square, semi-circle, and v-shape. I also included four environment scenarios: empty, choke point, scattered field, and wall. Each scenario placed circular obstacles in different arrangements, and the drones reacted according to the avoidance logic.

The app also included an attack mode. After placing an enemy, pressing “Attack” sent a choosen percentage of drones directly to the enemy. If any of these drones reached the enemy, they were removed from the simulation and the drone count decreased. After the attack, the remaining drones reorganized into a formation.

Finally, the telemetry plot showed the average distance of drones from their targets. This usually increased suddenly when a new formation was applied and then decreased again as the swarm stabilized. This gave me a simple way to monitor the performance of the swarm.

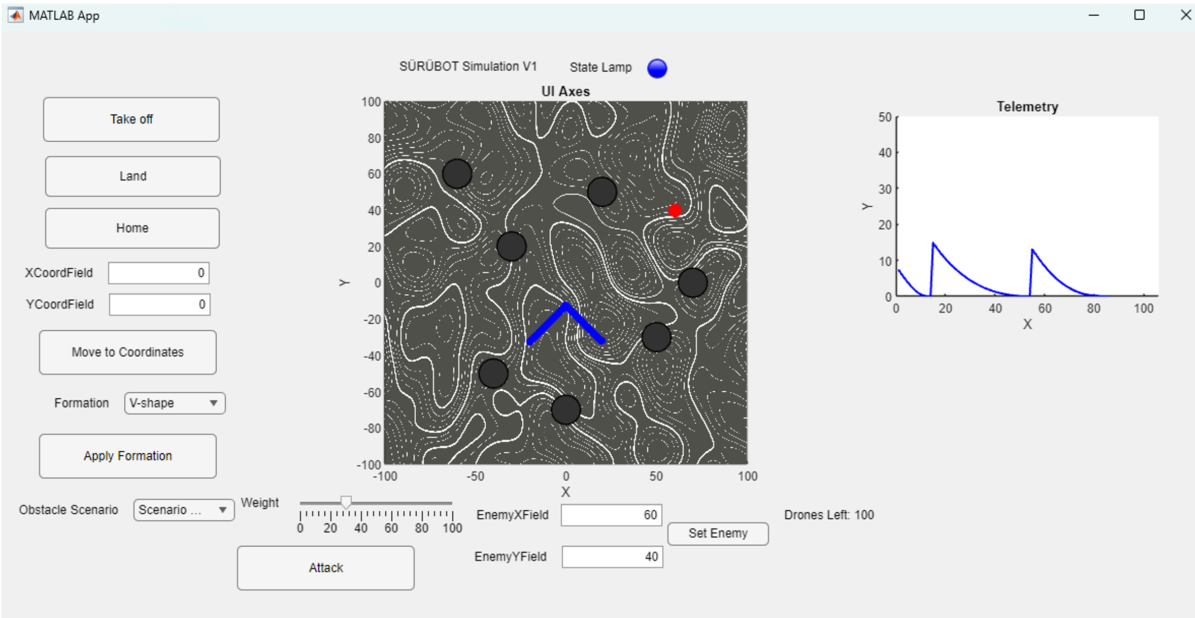

Figure 2. MATLAB UI

4.5.2 Results in MATLAB

I tested several cases in the MATLAB app. When changing formations in an empty field, the drones moved to their new positions and the error plot showed a peak followed by stabilization. In the choke point scenario, the swarm was able to pass through the narrow passage, with drones sliding around the obstacles thanks to the perpendicular force. In the wall scenario, drones recognized that they could not go straight and instead moved sideways until they found the gap. The scattered field created more irregular paths, but still no collisions occurred.

The attack demo also worked as intended. For example, when I set 30% of the drones to attack an enemy, they flew straight toward it, and once they arrived they were removed from the map. The remaining drones returned to a normal formation. The counter label updated correctly to show the new drone count.

These results showed that the basic combination of target forces, repulsion, and a danger zone rule was enough to simulate swarm behaviors and obstacle avoidance in MATLAB.

4.5.3 ROS 2 Environment

After finishing the MATLAB stage, I moved the project to ROS 2. In this stage I created two nodes: swarm_manager and city_visualizer.

The swarm_manager node was responsible for the drones. It published a MarkerArray on the topic /swarm_markers, where each drone was shown as a green sphere. It also subscribed to three topics. The first was /formation_command where I could send a string message with either "circle" , "square" or "v." The second was /move_goal, which allowed me to move the swarm's target center by publishing a point. The third was /obstacle_positions, where obstacle poses were received. The node had parameters for number of drones, step size, safety radius, and obstacle and drone weights. It updated the drone positions and republished them every 50 ms.

The second node, city_visualizer, published simple environment objects. It drew buildings (cubes), trees (spheres), and cars (small cubes). These were also published on /swarm_markers, so both drones and city objects appeared in the same RViz display. In addition, it sent obstacle positions on /obstacle_positions, which were used by the swarm manager.

I also wrote an ObstacleAvoider helper, defined in obstacle_avoider.hpp. This class checks whether a candidate direction is safe. It samples several forward points and rejects those that come too close to obstacles or other drones. It uses weights for obstacle distance and drone distance. This makes the swarm safer, because it avoids going into collisions.

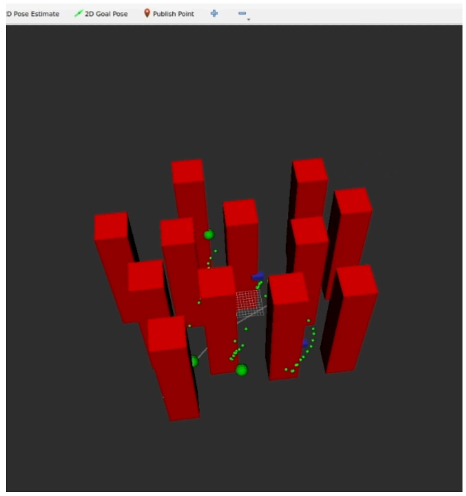

Figure 3. Visualization of swarm drones moving and obstacles in RViz (green = drones, red = buildings)

4.5.4 Results in ROS 2

I ran several example cases in ROS 2. With both nodes running, RViz showed the drones as green spheres moving inside a small city with buildings, trees, and cars. When I published "circle" on the /formation_command topic, the drones organized into a circle. When I published "v," they reorganized into a v-shape. By sending a point to /move_goal, the whole swarm shifted to a new location.

The city visualizer obstacles were also published and received by the swarm manager. This allowed the drones to check their directions against obstacles using the avoider class. In the visualization, the swarm moved smoothly while respecting the city layout.

These tests showed that the ROS 2 nodes were working as expected. Even though the ROS 2 version did not include the attack or telemetry features from MATLAB, it was still a useful visualization of swarm behavior and an important step toward possible future hardware integration.

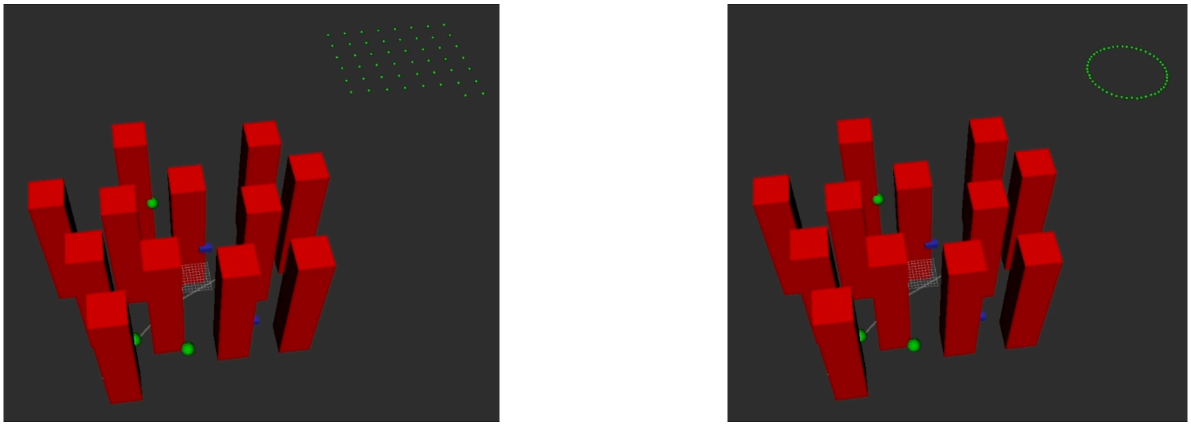

Figure 4. Swarm drones in RViz with building obstacles: square formation (left) and circle formation (right)

4.5.5 Comparison and Discussion

Both MATLAB and ROS 2 environments were useful in different ways. MATLAB was more interactive. It had four formation options, four obstacle scenarios, attack mode, and a telemetry graph. It was easy to test many cases quickly using the user interface.

The ROS 2 version was simpler but closer to real robotics. It only had circle, square and v-shape formations, and no attack or telemetry. But it allowed integration with other nodes, used standard ROS topics, and worked well with RViz. The addition of city objects made the visualization more realistic.

In general, the MATLAB stage was helpful for learning and prototyping, while the ROS 2 stage was important for connecting the swarm simulation to a real robotics environment. Together, these steps created a practical digital platform that supports the company's work in swarm technologies.

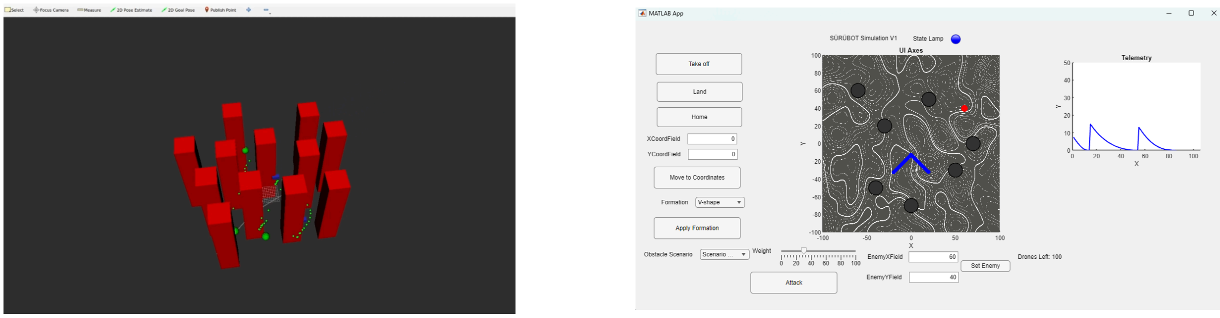

Figure 5. Comparison of the two simulation environments: ROS 2 with RViz (left) and MATLAB App Designer (right)

4.6 Results

The project resulted in two working simulation environments: one in MATLAB and one in ROS 2. Both of them achieved the main objective of giving the company a virtual platform to control a swarm and the environment together.

In the MATLAB app, I was able to show that the drones could take off, land, change between four formations, and avoid obstacles in different scenarios. The simple force model with repulsive and perpendicular terms was enough to keep the drones from colliding with obstacles. The attack mode also worked as expected: selected drones moved to the enemy and were removed when they reached it, and the rest of the swarm reorganized. The telemetry graph showed that the error decreased over time when drones stabilized into their new formation.

In the ROS 2 environment, I successfully visualized the drones together with a small city made of buildings, trees, and cars. The swarm_manager node responded to formation and move commands, and the drones could be displayed in circle or v-shape formations. The city_visualizer node published environment markers and obstacle positions, which were received and used by the swarm manager. Even though the ROS 2 version was simpler and did not include attack or telemetry, it showed that the concept from MATLAB could be transferred into a robotics framework.

Overall, the results confirmed that a virtual simulation environment can support the company's swarm research. MATLAB provided a fast and interactive prototype, and ROS 2 gave a realistic integration with visualization tools. Together, these results reached the project goals and created a base that can be extended further.

5. Internship Experience

5.1 Learning

During my internship I learned how the behavior of drones, which are always moving and changing with time, can be transferred into software. I saw how swarm logic works in practice: how a group of drones can be managed together, how they keep formations, and how they avoid obstacles. I applied methods like attractive and repulsive forces, perpendicular forces, and danger zones, and I understood better how these techniques are coded and tested in simulation.

Because I was also placed in the Mechanical Department, I worked on some tasks there as well. In that part of the internship I used my knowledge of CAD tools, mainly SolidWorks and NX, to make the necessary designs and drawings requested by the team. These were smaller projects compared to the swarm simulation, but they helped me use and improve my mechanical design skills in a real company environment.

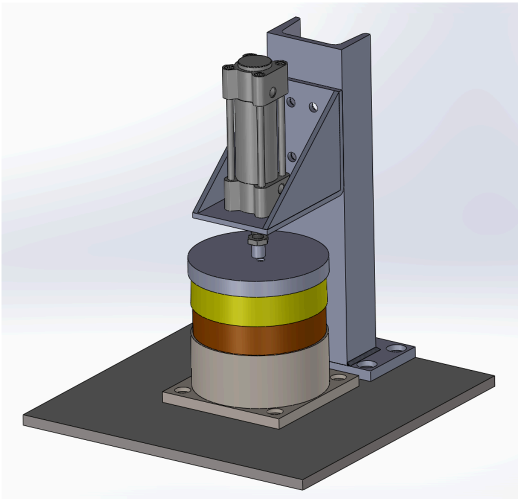

Figure 6. Fuel core piston (designed in SolidWorks)

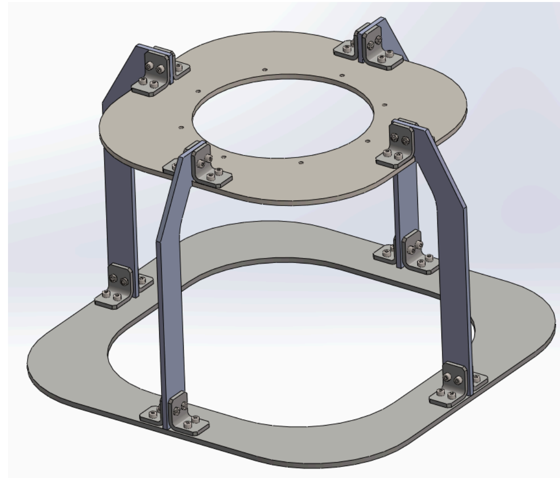

Figure 7. Fixture / Support Frame

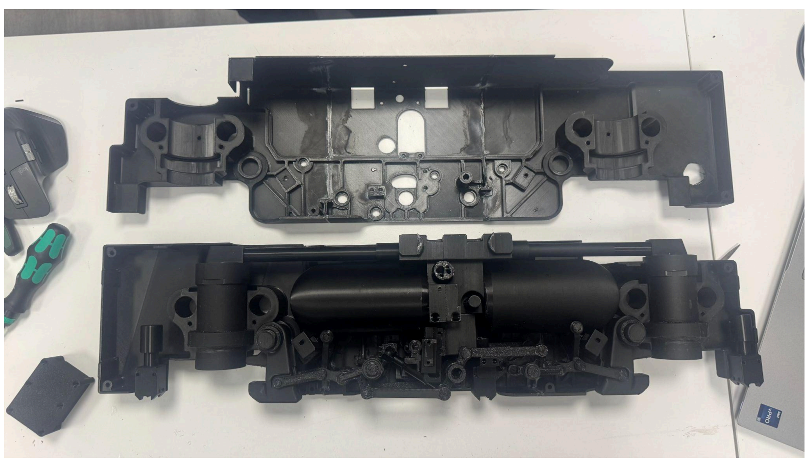

Figure 8. ERU (Electro-Mechanical Release Unit) 3D printed prototype for cabling

This experience also changed my view about my career. At first I thought I might focus more on the mechanical side of mechatronics, but after seeing the current trends and working in both areas, I realized that software and control are becoming more important. I found myself more interested in coding, swarm control, and artificial intelligence rather than in pure mechanical design. For this reason, I decided to concentrate my project and my future plans more on software and control systems.

5.2 Relation to Undergraduate Education

The work I did in this internship was directly connected to several courses I took at university. For example, the Artificial Potential Field (APF) method that I used in the swarm simulation was something I had already studied in class. In ME425 (Autonomous Mobile Robots) we worked with MATLAB codes and algorithms for robot navigation, and that background helped me a lot in coding the swarm behaviors and obstacle avoidance in MATLAB. Also, both ME303 and ENS206 gave me practice with MATLAB programming, so I was already comfortable with using the software for simulations.

In the Mechanical Department, my tasks were related to design, and there I used the knowledge I gained from ENS209 and ENS410, where I learned CAD tools. The experience with SolidWorks and NX from these courses was useful when I had to prepare parts and drawings during the internship.

One thing I noticed was that my courses at university were very strong in MATLAB and general robotics, but I felt that I was not equally prepared for working with C++ and ROS 2. I had to learn some of it by myself during the internship. This made me realize that having more C++-based content in the curriculum would have helped me, especially for projects like this one.

5.3 Difficulties

The most difficult part of my internship was actually related to the department I was placed in. For the two months of the internship I worked in the Mechanical Department, but I realized very quickly that I did not want to focus my career mainly on mechanics within mechatronics. This was a bit discouraging for me, because I understood that I am more interested in software and control. I could not officially move to another department, because there were no open positions, but instead I tried to balance this by taking a software-related project about swarm simulation. The company supported me in this choice and allowed me to work on the simulation project while still being based in the Mechanical Department. This way I could connect with the Control, AI, and Software teams, and it helped me use my time better.

Another difficulty was that I had to work with ROS 2 and C++, which I did not learn much in my courses. At first this was challenging, because I was more experienced in MATLAB. But by reading documentation and asking engineers small questions, I was able to make progress and get the nodes working. In the end, this challenge was also a learning opportunity.

Apart from these points, I did not face very big problems. The company and the engineers were supportive, and they helped me whenever I needed guidance.

6. Conclusions

During my internship at Altınay Defense, I had the chance to work on both a personal software project and several tasks in the Mechanical Department. My main project was to design a virtual environment for swarm drones. I first created an interactive app in MATLAB where I implemented swarm formations, obstacle scenarios, and an attack mode. Later, I transferred the idea into ROS 2 and RViz, where I published drones and city objects as markers and tested simple formation commands. This project helped me understand how swarm logic, obstacle avoidance, and simulation can be coded and tested in practice.

At the same time, since I was officially placed in the Mechanical Department, I also worked on some mechanical design tasks. Using my background in SolidWorks and NX, I prepared models and drawings for small projects requested by the department. This allowed me to continue practicing my mechanical design skills and support the team in their ongoing work.

From this combination of experiences, I reached some important conclusions. First, I realized that my interest is much stronger in the areas of software, control, and autonomy compared to purely mechanical work. The simulation project gave me confidence that I can contribute more effectively in these fields. Second, I also saw that having a mechanical background is still valuable, because it helped me communicate better with the team and understand the physical limits of the systems. Finally, I understood that internships are not only about practicing skills but also about discovering what directions I want to follow in my career.

Overall, this internship confirmed to me that I want to focus on software and control systems in the future, while still supporting myself with mechanical knowledge when necessary.

7. Recommendations

Based on my internship experience, I would like to share a few recommendations.

For future interns, I recommend trying to communicate with engineers from different departments. Even if you are officially placed in one department, it is very useful to see how mechanical, control, AI, and software teams all work together. This helps to understand the full picture of a project. I also suggest that interns spend time learning both MATLAB and ROS 2, because these tools are widely used in robotics and are directly connected to the type of projects done in the company.

For the company, I think it would be valuable to give interns more flexibility in choosing projects according to their interests. In my case I was placed in the Mechanical Department, but I was lucky to also work on a software project. This was very helpful for me, and I believe it would be useful for other interns as well.

Finally, I recommend that the company continue to support projects that combine different disciplines. Swarm simulations are a good example of how software, control, and mechanical design all come together, and I think these kinds of projects are great learning opportunities.

8. References

Marek, D., Biernacki, P., Szyguła, J., Domański, A., Paszkuta, M., Szczygieł, M., Król, M., & Wojciechowski, K. (2025). Collision Avoidance Mechanism for Swarms of Drones. *Sensors*, *25*(4), 1141. https://doi.org/10.3390/s25041141

Tang, Ruipeng & Tang, Jianrui & Talip, Mohamad & Kumar, Narendra & Xu, Xifeng. (2025). Enhanced multi agent coordination algorithm for drone swarm patrolling in durian orchards. Scientific Reports. 15. 10.1038/s41598-025-88145-7.

STM Savunma Teknolojileri Mühendislik ve Ticaret A.Ş. (2020). *Sürü İHA sistemleri raporu*. STM ThinkTech. Retrieved from https://thinktech.stm.com.tr/tr/gelecegin-harekat-ortaminda-suru-dronelar-ve-insansiz-sistemlerin

-rolu

Corke, P. (2017). *Robotics, vision and control: Fundamental algorithms in MATLAB* (2nd ed.). Springer.

Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to autonomous mobile robots* (2nd ed.). MIT Press.